ThreadUD

Project Prototype documentation

Developed by

Nikita Ciuciu – B00152278

David Thornton – B00152842

Julia Udvari – B00153580

17/12/2024

Contents

1. Overview of System 4

2. Use-Cases 5

Use-Case Specifications 5

1. User Registration 5

2. User Login 7

3. View Home Page 8

4. Join a Thread 9

5. Create a New Thread 10

6. Make a Post 12

7. Like a Comment 13

8. Comment on a Post 15

9. Search for Threads 16

10. Update Profile 17

11. Receive Push Notifications 19

12. Log Out 20

13. Moderator Manages Posts 21

14. View my posts 23

15. View my comments 24

16. Reply to a comment 25

17. Like a post 26

18. Share a post 27

19. Forgot password 28

Use-Case Diagrams 30

3. System Design 33

High-end detailed Wireframes 33

System Diagrams 35

System Design Implementation 36
Already implemented (1st semester) 36
Yet to implement (2nd semester) 37
4. Detailed project plan for remaining work 39
Timeline for remaining work 39
Task allocation breakdown 39
5. Contributions (for this document) 42

Table of figures

Figure 1 - User Account Management Use-Case diagram 30
Figure 2 - Thread and Post Interaction Use-Case diagram 30
Figure 3 - Engagement and Interaction Use-Case diagram 31
Figure 4 - Search and Notification Use-Case diagram 31
Figure 5 - Moderator-Specific Actions Use-Case diagram 32
Figure 6 - User Content Management Use-Case diagram 32
Figure 7 - Home & Profile pages Wireframes 33
Figure 8 - Thread & Posts pages Wireframes 34
Figure 9 - ERD diagram for Mongo Database 35
Figure 10 - Class Diagram 35

1. Overview of System

The goal of the project involves creating an online community space. This space will foster student interconnection and cooperation. The focus is on academic themes related to coursework and module study. The application is set to function as a community, where students can post queries and questions. Peers can offer feedback and provide suggestions.

The aim is to resolve the lack of a dedicated platform to let students easily access academic advice from other students. The application also encourages academic conversations which could be outside normal college hours. These specific conversations benefit students in a way that allows deeper understanding, collaboration and solving problems together. It is advantageous for continuous learning, pushing students to stay engaged and motivated.

The app will create a supportive student community by providing a platform where students can interact and help each other. It will also foster interesting student interaction. The app offers quick access for peer-to-peer assistance and in return this access promotes learning.

This project centres around development of a mobile application, it will have multiple features to support academic team effort.

Users will be able to post their queries regarding certain university modules, which will be sorted into threads or categories. This simplifies the way students find and offer input to relevant course discussions.

User experience will be greatly enhanced by the app. This will include various account set-up and management basics. These basics will allow students to follow particular modules. Users may also follow certain threads based on their interests.

The app will also feature push notifications to keep users with what is happening in their communities and modules. They will be provided with this data within the modules they monitor. The app will feature a real-time chat attribute which will allow students to be part of live conversations where they can add their input immediately.

Deliverables for this project include formation of fully functional mobile app with a user-friendly interface and a robust backend that will support data and user accounts. Moderation features will be added to ensure that discussions stay relevant and helpful to course material, and ensure a safe learning environment for all students. This project will also include testing to ensure the app's effectiveness and user friendliness followed by documentation which will keep the examiners informed about our project and aid the team in future maintenance of the app.

During this stage, advanced features like private messaging and complex recommendation algorithms will not be included, as the focus needs to remain on the main platform functionality and the project timeline won't allow for the implementation of such complex features.

2. Use-Cases

Use-Case Specifications

1. User Registration

Description: A new user should provide account details such as name, email, password, course, and year in which he is studying

Actors: New user (a Student who has not signed up yet)

Preconditions:

- User has the installed app on his/her mobile and opens the same.
- User should not already have been registered on ThreadUD

- The user must be a currently registered student at TU Dublin

Postconditions:

- The details pertaining to the user account are created and stored in the system database
- User is either taken to the login page for signing in after successful registration

Main Flow:

- The customer reaches the "Register" option on the homepage or tries to create/join a thread or even comment on a post, after which he will be forwarded to the Register page
- The application opens a register form and asks the user to provide their name, email, password, course, and year of study
- The user fills in the registration form and submits it
- The application checks the validity of the form fields, such as email format or password strength, for instance
- The user details get accordingly stored in the database if the above validation passes
- A success message is displayed while the user is either forwarded to the login page or automatically logged in

Assumptions:

- The user will have a stable Internet connection
- A valid email address can be inputted with a strong enough password
- The system should keep users' data securely, especially the password
- This can be possible by using an encrypted password

2. User Login

Description: A registered user who has logged in accesses greater functionality of the system using their e-mail and password

Actors: Registered user

Preconditions:

- The user has already created an account and has their login credentials (email and password)
- The login form must be available

Postconditions:

- User is successfully authenticated and then logged into the system
- The user is then taken to the home page, where account restricted features are accessible, such as posting, joining threads and commenting on other's posts

Main Flow:

- The user navigates to the homepage and clicks on "Login" or is redirected to the login page when trying to use an account restricted feature
- The app will provide a login screen that asks the user for their email and password
- In that, the user fills in the appropriate fields and submits the form
- The app verifies the provided credentials against the stored data
- And if the login is successful, then the user is provided access and he is taken to the home page
- In case of failed login, because of an incorrect password or email, it throws an error message and the user is prompted to retry

Assumptions:

- It assumes that the user remembers his login credentials or at least has them handy
- The system has encryption and security features that ensure credentials of users are kept safe
- The Internet connection must be stable to allow authentication

3. View Home Page

Description: Both a logged-in user and a guest can view the home page and has the ability to view all posts on threads. Guests can view threads but would need an account for interacting on the site to make a post or join a thread

Actors: Any user, either guest or logged in

Preconditions:

- The user opens up the app, whether they were logged in or not
- The home page and its contents are accessible regardless of the user's login status

Postconditions:

- Users who sign in will have additional options to join threads or make posts
- Guests can view and search threads, but they will be requested to log in or register if they make an attempt to interact with the content

Main Flow:

- User goes to the app and is taken directly to the home page
- On the home page, it lists a variety of threads related to the modules and their posts

- It enables the user to either scroll through posts or use a search bar for any particular thread
- If a user clicked into a thread, they would then be able to view all the posts within a given thread
- If the user attempts to enter the thread or make a post, the app checks whether he is logged in
- What they can do is proceed with their actions if they are logged in
- In the case of not being logged in, users are invited to log in or register

Assumptions:

- Home page content (Threads and Posts) are publicly viewable
- The application also provides a search feature, which can be used both by guests and by users who have logged in
- It has to be connected to the internet to show the threads and posts in real time
- For every logged-in and guest user, there should be a variation in UI elements, such as a button or an option, depending on his/her log-in status

4. Join a Thread

Description: A logged in user can join the thread on any particular module for discussion

Actors: The logged in user

Preconditions:

- The user has to be logged in
- The thread for the respective module or topic has to be available

Postconditions:

- The user becomes a part of the thread and is considered as a participant to it
- They can view or make a post in that thread

Main flow:

- The logged in user goes directly to either the home page or the search page in order to locate a thread
- The user selects a thread they would like to join
- The application checks whether the user is logged in
- If they are logged in, the application presents an option to 'Join Thread' this could be in the form of an icon or button
- The user clicks the 'Join Thread' button
- The backend adds this user as a participant of the thread update in the database
- This thread now appears under the user's joined threads list
- The user now can actively participate in the posting or commenting of this thread

Assumptions:

- User has already logged into the app and finished the registration
- The application will have the capability of tracking user membership within threads, meaning a user can be part of more threads
- One thread should have the ability to let multiple users in without performance degradation
- The internet connection should also be stable enough to support real-time interaction

5. Create a New Thread

Description: The logged-in user opens a new thread for the module or topic that does not have any thread open yet

Actors: The logged-in user

Preconditions:

The account should have been logged in

The thread of the particular module and topic does not exist

Postconditions:

A new thread has been opened and other users can see all threads, whether a guest or a logged-on user

We can immediately join the creator to this thread as the first participant in this thread

Main Flow:

- The user has logged in and arrived at the "Create Thread" page
- The application requests that the user provide input for the details of the new thread. Among these are:
    - Module or topic name
    - Thread description/rules
- The user fills up his/her information and then submits the form
- The app validates the input, meaning for instance that the thread title is not blank and that there does not already exist a thread with that title
- If the input is valid, the app creates a new thread in the system's database
- It will now appear on the home page for all users to see
- The creator of the thread automatically joins the thread
- A success message should be displayed, and the user should be routed to the page of the newly created thread

Assumptions:

- The user is already logged in
- First of all, the system checks if there was already a thread on the same topic/module so that there are no multiple threads related to the same topic
- The system shall possess the necessary back-end infrastructure required to create, store, and display threads to the users
- This operation must take place with direct Internet access to perform the creation or update of a thread in real time

6. Make a Post

Description: A logged-in user can post in a thread to ask or share any information that is related to the module

Actors: Logged-in user

Preconditions:

- The user is logged in
- The user has joined a thread in which he/she wants to post

Postconditions:

- The post gets successfully submitted and is visible to other users within the thread
- The post could also be read by logged-in users, including guests, but it could have comments only from this logged-in user

Main Flow:

- The logged-in user navigates to the thread where they want to create a post

- The application provides the user with a text box or more correctly, an input field where they'll be able to write their post
- The user creates their post in a text box, writing the post text, links, and other forms of media if the application allows them to do so
- The user submits the post by clicking a "Submit" or "Post" button
- The application checks the content of the post for example, for an empty submission or for rule-breaking content
- If the post is valid, then the app saves it into the database and visualizes it within the thread
- Other users can view that post within a thread now
- A success message is shown with confirmation that the post has been made

Assumptions:

- A user has already logged into the app and joined the thread
- The system is capable of storing and displaying posts in real time to all users
- The app may flag certain posts through a moderation system for not being appropriate
- The internet connection is good, thus the post can be submitted and shown

7. Like a Comment

Description: Like a comment in a thread of a post, to show approval or agreement with what has been said

Actors: The logged-in user, initiating the like

Preconditions:

- The user has to be logged in to their account
- The comment they decide to like should, of course, be present in a thread

- The user has not liked this comment in the past to avoid the duplication of likes from the same user

Postconditions:

- A like is sent to the comment, which refreshes the comment's like count
- A record of the user's like is saved in the database to ensure they can't like the same comment more than once
- Other users are presented with the refreshed like count for that comment

Main Flow:

- The user logs in and goes onto a thread, they see different posts and comments
- The user sees the comment they want to like
- The user clicks on the "Like" button/icon next to the comment
- The system checks if the user has liked the comment or not
- If it is a like that has not taken place, update the like count and store the like in the backend (database)
- The like count of the comment is updated and presented to all users who can see the thread
- Provide feedback to the user, such as a small visual update or short message, for a successful action
- If the user tries to like the comment again, the app prohibits this action (optional)

Assumptions:

- The user has logged in to the application and does have permission to interact with comments
- The application has a backend infrastructure to store and manage likes, ensuring no user can like a comment more than once
- The app's frontend has been set to properly display the count of likes and update them in real time

- Internet connectivity is required to update the like count and reflect changes for all users
- Nikita & David will take care of the backend side of this feature, implementing the like interactions correctly and making sure it is reflected properly in the database

8. Comment on a Post

Description: Current user comments an existing post in a thread as a response or feedback or further input

Actors: User is logged in

Preconditions:

- The user has been logged in
- The user joined the thread from which the post exists
- There is a post existing in the thread

Postconditions:

- The user's comment has been added to the post and thus is viewed by other users in the thread
- The thread has been refreshed and the comment appears under the post for view and further interaction

Main Flow:

- The user having logged in, opens the thread in which the post on which he wants to comment is present
- In most cases, after clicking the post, the user sees an option to add a comment a text field

- It is here that the user types in his comment in the provided input
- The user clicks the "Comment" button
- The app provides some kind of input validation: an input check against void or inappropriate comments
- Once it's been successfully validated, that comment gets added, through the app, to the post in the database
- The comment should come under the posting under the thread and now can be viewed to be commented on by the other users

Assumptions:

- The user must be logged in with the permission to comment on this thread.
- Because it is this kind of system, the content may refresh in real time, so comments should appear instantly once posted
- Internet connectivity is sufficient for a user to post comments and display them
- The system may include moderation or filtering against comments with inappropriate content

9. Search for Threads

Description: A guest or logged-in user shall search for threads by module names, topics, or keywords

Actors: Any user may use the system as a guest or a logged in user

Preconditions:

- The user shall have access to the app's search feature
- Threads and posts must be pre-existing within the system

Postconditions:

- The result of the search lists threads relevant to what the user inputs
- Each of these results will be clickable to thread view with posts

Main Flow:

- The user enters the search functionality: this could be through a search bar on the home page or through a dedicated search page
- The user types their query in the search bar: this could be the name of a module, a key word, or even a topic
- The user submits the search query
- The app will query the database for threads that match the entered search terms
- View search result, where the page enumerates all thread titles and short descriptions
- Any of these results can be clicked by the user in order to view the full thread

Assumptions:

- The search facility in the app is available to both guests and logged-in users who have accessed it
- The search algorithm of the app is capable of matching threads by keywords, module names, and relevant topics
- It supports stable Internet connectivity for immediate searching and retrieving outcomes
- It can allow different search queries to attain their effective and relevant results

10. Update Profile

Description: A logged user would be able to edit his profile, changing his name, email, course, or year of study

Actors: Logged-in user

Preconditions:

- The user must be logged in to their account
- The user's profile information should already be present in the system, ready to be modified

Postconditions:

- User profile is updated with the new information, saved to the database
- The profile data is shown in other areas of the app where the profile info may be used, such as when making posts

Main Flow:

- User opens the profile page
- User chooses the option "Edit profile”
- The existing profile information is displayed
- The user can change any of the following fields: Name, Email, and Course or year of study
- The user changes the desired fields and submits it
- The app validates the fields entered by the user for example, checking if the email submitted is in a valid format
- On validation, the application updates the profile data in the database
- After posting a success message, this updated profile shows up everywhere in the application, such as in posts or thread interactions

Assumptions:

- The user can view his profile after opening an account in the app

- The system shall be capable of checking input provided by the user and controlling the integrity of data, including the format of email addresses
- The system shall be properly secured for the handling of sensitive information such as emails or passwords
- Internet shall be used to store updated profile data into a backend database

11. Receive Push Notifications

Description: Through the application, users receive push notifications whenever there are new posts or comments within threads they are following or participating in

Actors: User logged in

Preconditions:

- The user must be logged in and allowing push notifications for the app
- The user has to join or follow some threads to get updates on the same
- The app must have notification access permission by the user

Postconditions:

- The user's device provides a notification when there is new activity in the user-followed thread
- There is also a displayed clickable link, which leads to the relevant thread or post for the user

Main Flow:

- The user grants permission for push notifications either at the prompt or through the settings within the app
- The user interacts with or follows certain threads

- If there is a new post or comment within a followed thread, it will initiate a push notification via the backend system
- The push notification is delivered to the user's device
- From here on, the notification will appear for the user on their device, accompanied by a short message like "New post in [thread name]"
- The tap on the notification opens the app and forwards the user to the relevant thread or post

Assumptions:

- The user has allowed the settings, including push notifications from their device for this application
- The application has some backend functionality regarding push notifications and triggers the push notification on events such as a new post or reply
- The system should be able to send numerous notifications without overwhelming the user
- The internet connection is good enough for sending and receiving to be in real time

12. Log Out

Description: A logged-in user logs out successfully from their account, resulting in the close of the current session

Actors: User logged in

Preconditions:

- The user has already signed in
- There is a Logout option available in the application UI

Postconditions:

- User is logged out of his or her account, and the session is terminated; user is taken back to the log-in page or home page where posts can be viewed as a guest

Main Flow:

- The user opens the application and directly goes to the account or settings section
- There, he presses the "Log Out" option
- The application asks for confirmation from the user regarding logging out
- The app kills the session and logs out the user upon the positive confirmation by the user
- The user will be taken to the login page or home page where they can further browse as a guest, without being able to post or join threads
- A confirmation message may be displayed for successfully logging out

Assumptions:

- The user is considered to be already logged in
- An easily accessible log out option is provided by the app
- The system correctly clears the session and ensures the user is securely logged out
- In case of a logout, the session token or authentication data of the user is marked invalid to avoid unauthorized access
- Log out might need internet connectivity in order to log the user out from the system

13. Moderator Manages Posts

Description: A moderator reviews, edits, or deletes posts or comments that do not comply with the community guidelines of the application

Actors: Moderator (a user with special privileges)

Preconditions:

- Moderator is authenticated
- Moderator has special admin privileges to manage posts and comments
- Some posts or comments are flagged for moderation or need a review

Postconditions:

- The moderator inquires and modifies posts or comments
- Based on what is done, the post or comment may be edited, deleted, or flagged
- He notifies other users if need be (e.g., the post is deleted and the user informed)

Main Flow:

- The moderator logs into the app and proceeds to the moderation dashboard, if there is one, or sees posts flagged within threads
- The moderator will select the post or comment to be moderated, which could be in response to a user flagging or a manual selection
- The app then displays different moderation options that include editing, deleting, setting for review, and giving a warning to a user
- The moderator makes a choice of action (say, to delete a rude comment or change a post)
- The action is performed, and the database is updated
- Users on whom this action has an impact can be informed by some kind of notice, e.g., "post deleted" or "warning sent"
- The thread or post is updated to all users with respect to the moderation action

Assumptions:

- A moderator is somebody with special rights who can moderate entries made in the system
- The system provides different tools for moderation and facilities for easy recognition and identification of problem posts
- The app enforces community guidelines and brings to light inappropriate content through measures in place for its detection or flagging
- One needs to have a stable internet connection where any changes by the moderator appear in real time
- The app may store logs or history of moderator actions for audit reasons

14. View my posts

Description: A user views all the posts they have made across all threads.

Actors: User, posts, threads(name)

Preconditions:

- User has an account
- User is logged in

Postconditions:

- System has displayed all the user's post if they have any

Main Flow:

- The user will select the profile button to navigate to the profile page.
- The user will select the “My posts” button

- The system will load all the posts made by the user with thread details
- The system will display all the posts created by the user if they have made posts, otherwise a “Nothing to see here!” message will be displayed

Assumptions:

- The user will have a stable Internet connection
- The user has an account
- The user has made posts
- The system can retrieve the users posts

15. View my comments

Description: A user views all the comments they have left on posts, with the post title, and all replies to others comments, with the comment they replied to.

Actors: User, comments, post, thread, replies

Preconditions:

- User must have an account
- User must be logged in

Postconditions:

- System has displayed all the user's comments

Main Flow:

- The user will navigate to the profile page.
- The user will select the “My comments” button

- The system will load all the comments made by the user with post and thread details
- The system will display all the comments created by the user and the title of the post it was left on if they have made comments, otherwise a “Nothing to see here!” message will be displayed

Assumptions:

- The user will have a stable Internet connection
- The user has an account
- The user has made comments
- The system can retrieve the users comments and comment information

16. Reply to a comment

Description: A user replies to a comment on a post, or to another reply.

Actors: User, comment, post, thread

Preconditions:

- User must be logged in
- User must be at a posts post page

Postconditions:

- The system will add the comment information to the database
- The system will display the new reply to the user under the comment/reply

Main Flow:

- The user will navigate to the comment they want to reply too

- The use selects the reply button
- The system will display a textbox for the user to enter their reply
- The user will enter their reply and press submit
- The system will add the reply to the users view of the post page
- The system will update the database with the new reply information

Assumptions:

- The user will have a stable Internet connection
- The user has an account
- The system can retrieve all the comments under a post
- The system can update the replies to a specific comment to include the new comment

17. Like a post

Description: A user will like a post

Actors: User, post, thread

Preconditions:

- User must be logged in
- User must be on the thread page of the post or the post's page

Postconditions:

- The system will update the users view to display the post as liked
- The system will update the database to increment the posts like count and add the post to the users likes

Main Flow:

- If the user is on the thread page of the post, they will navigate to the post they want to like
- The user selects the like button
- The system will display the post as liked to the user
- The system will update the database, incrementing the post's like count and add the post to the users likes

Assumptions:

- The user will have a stable Internet connection
- The user has an account
- The system can retrieve all the posts information
- The system can update the users who liked the post to include the new user

18. Share a post

Description: The user will share a post with another user (through another social media app)

Actors: User, post, thread

Preconditions:

- User must be logged in
- User must be on the thread page of the post or the post's page
- User must have another app to share post through

Postconditions:

- The user will share the post to another user through another social media platform

Main Flow:

- If the user is on the thread page of the post, they will navigate to the post they want to like
- The user selects the share button
- The system will display a list of possible apps the user can share the post through
- The user will select the app they want to share the post through
- The user will be redirected to the selected app to choose who they will send it too

Assumptions:

- The user will have a stable Internet connection
- The user has an account
- The user has other social media apps to share the post through
- The system can retrieve that post information
- The system can create the link to be sent with the post information

19. Forgot password

Description: If a user forgets their password when logging in they can reset it

Actors: User

Preconditions:

- User must have an account
- User must be logged out of the app

Postconditions:

- The user will be logged back into the app
- The system will update the database with the users new password

Main Flow:

- The user will enter the app and select to log in
- The user will select the forgot password button
- The system will direct them to a page to enter their email address
- The user will enter their email address
- The system will generate a random 6 digit code and send it to the users email
- The system will display an input box and ask the user to input the code
- The user will enter the code form the email
- If the user enters the wrong code the password reset will terminate and the user will be redirected to the login page
- If the user enters the correct code the system will display a text box for the user to enter their new password
- The user will enter their new password
- The system will log the user in and direct them to the home page
- The system will update the users password in the database

Assumptions:

- The user will have a stable Internet connection
- The user has an account
- The user enters the correct email address
- The user still has access to this email account
- The system can send code to the users email account
- The system accepts the correct code
- The system can update the users new password

Use-Case Diagrams

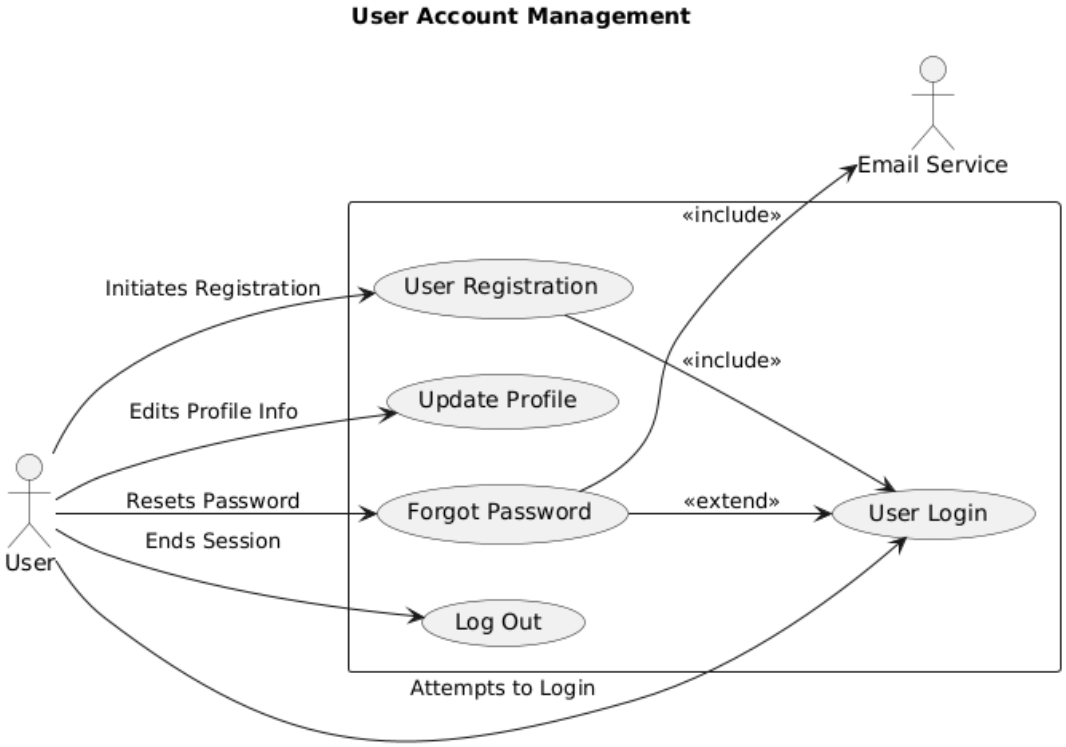


Figure 1 - User Account Management Use-Case diagram

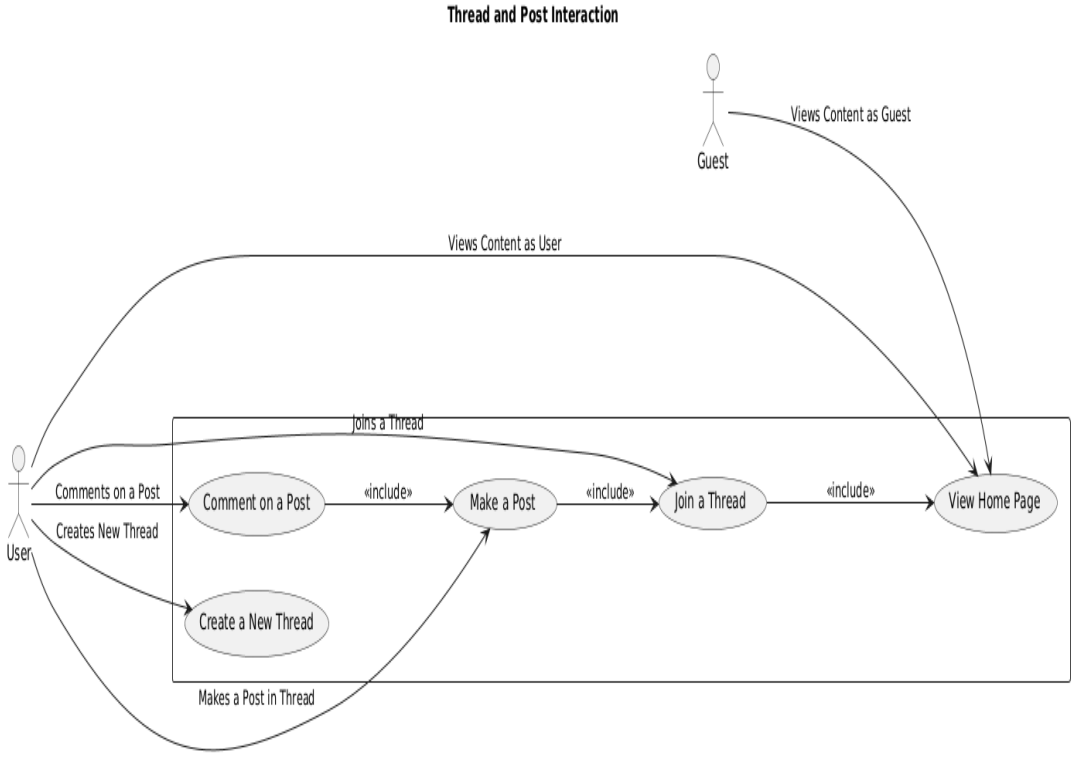


Figure 2 - Thread and Post Interaction Use-Case diagram

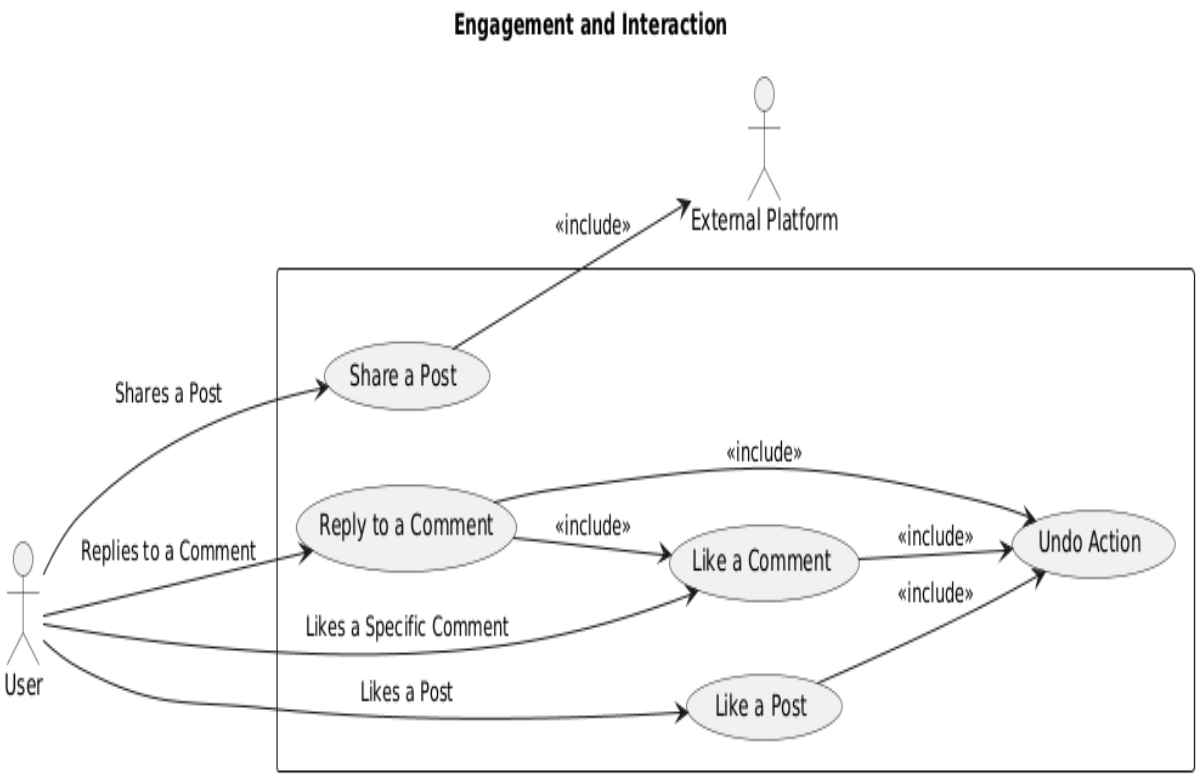


Figure 3 - Engagement and Interaction Use-Case diagram

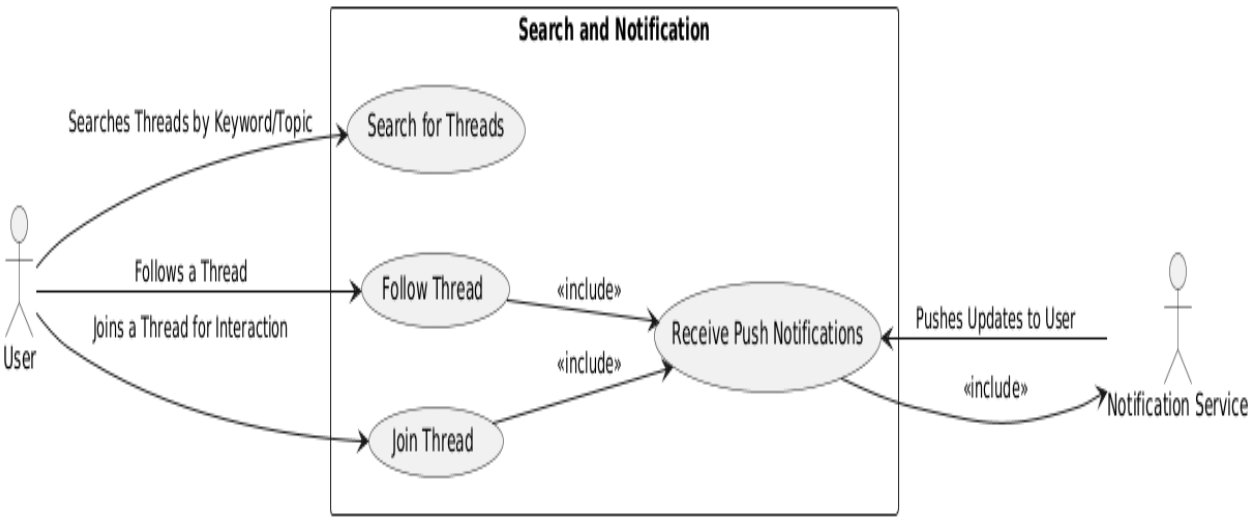


Figure 4 - Search and Notification Use-Case diagram

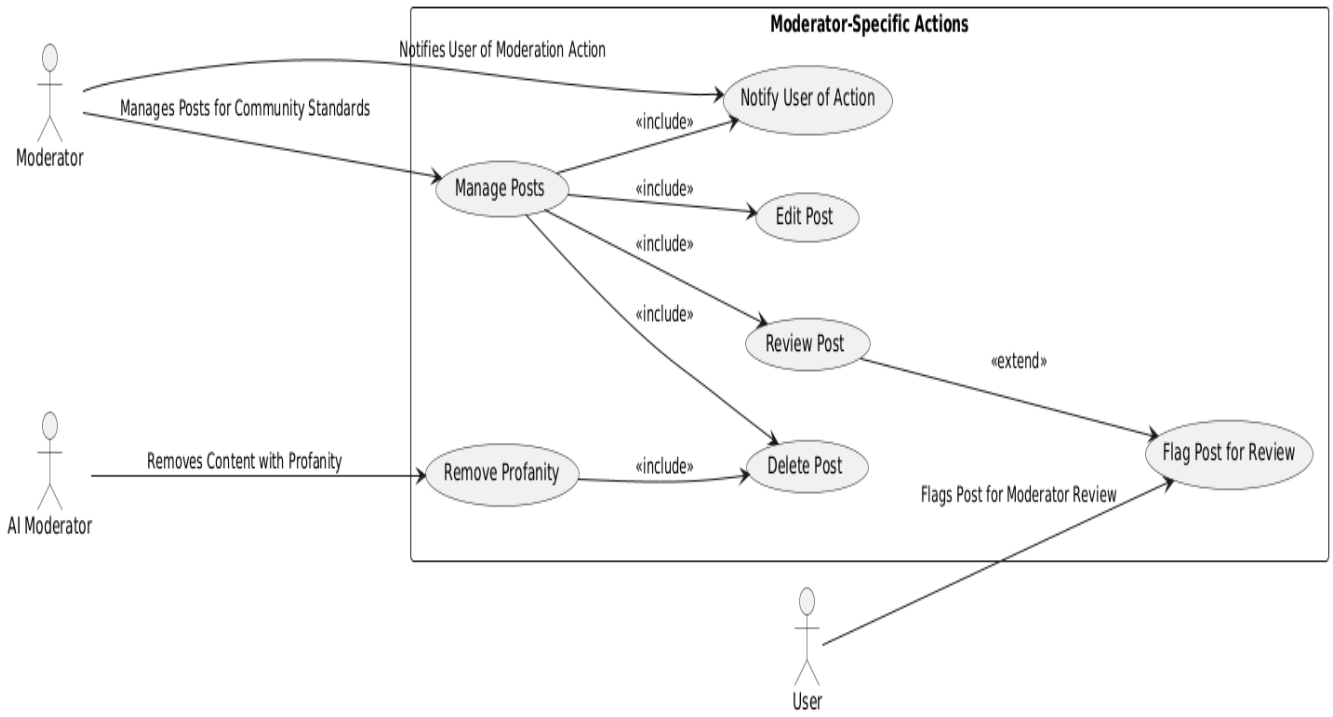


Figure 5 - Moderator-Specific Actions Use-Case diagram

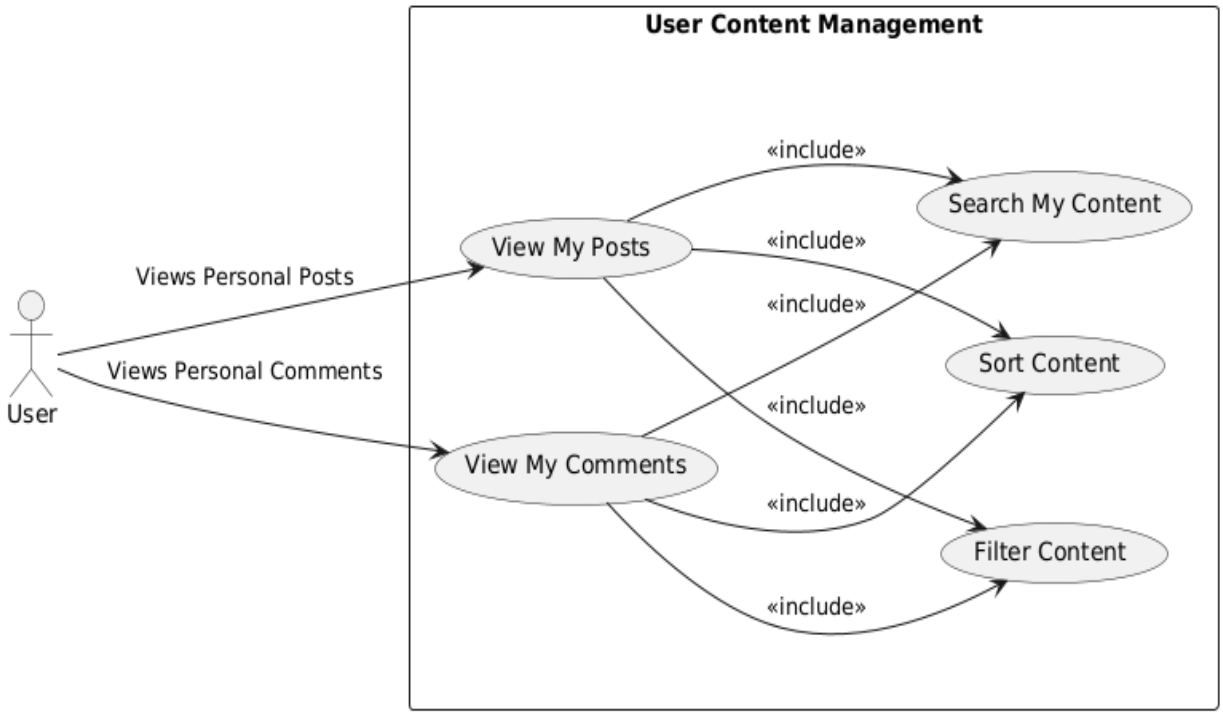


Figure 6 - User Content Management Use-Case diagram

3. System Design

High-end detailed Wireframes

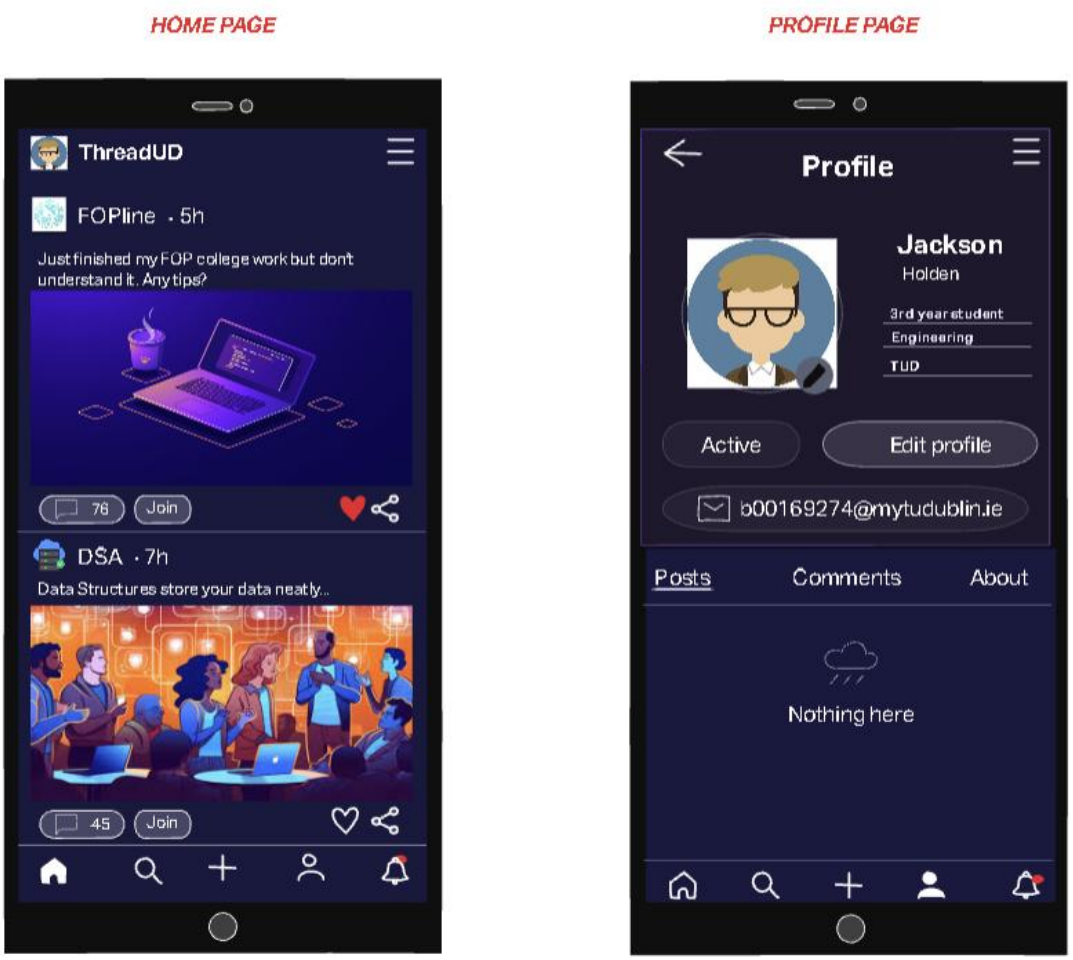


Figure 7 - Home & Profile pages Wireframes

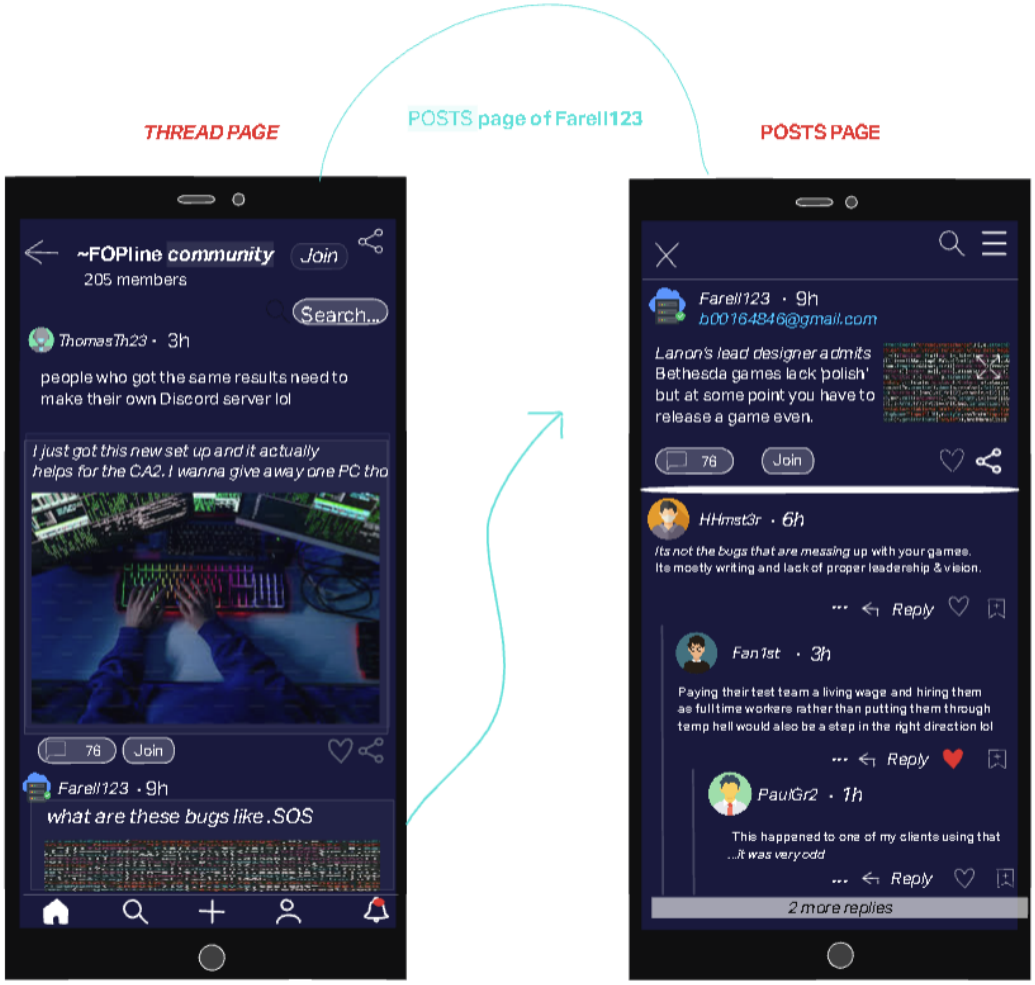


Figure 8 - Thread & Posts pages Wireframes

System Diagrams

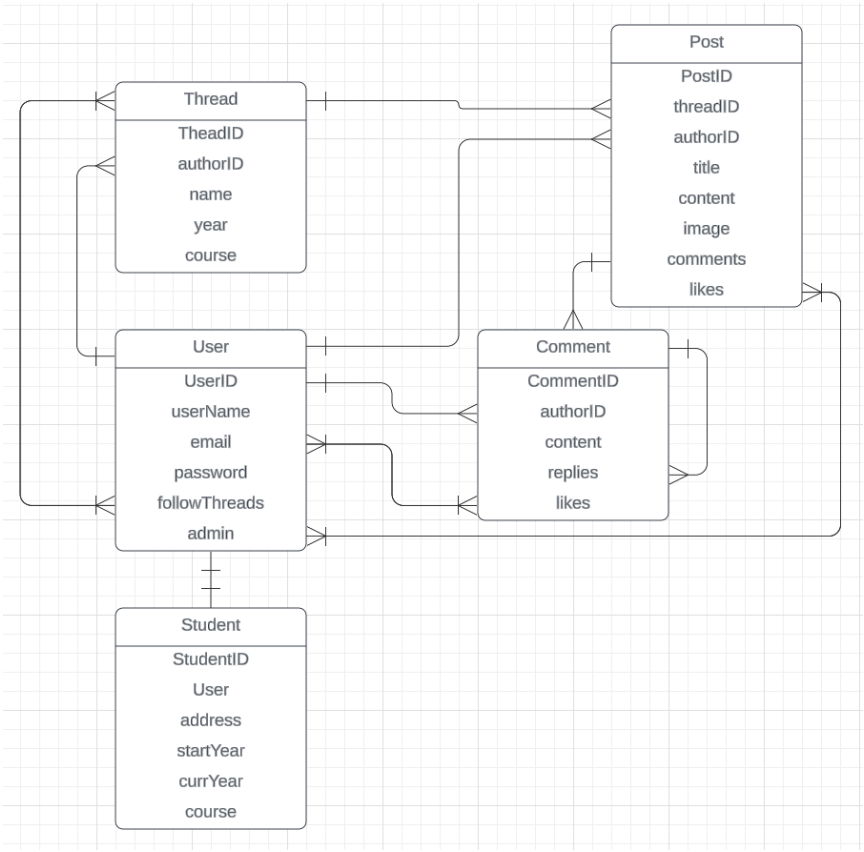


Figure 9 - ERD diagram for Mongo Database

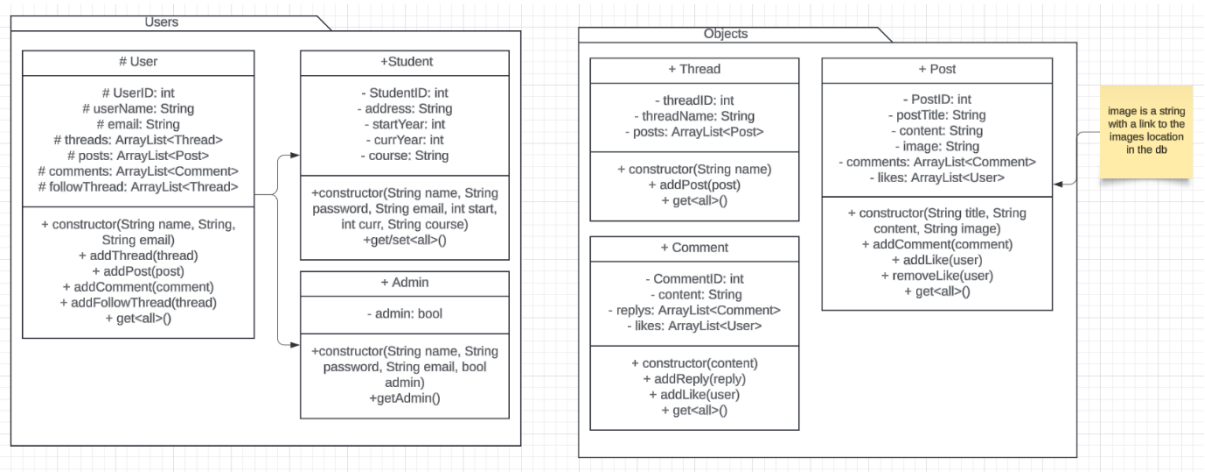


Figure 10 - Class Diagram

System Design Implementation

Already implemented (1st semester)

Home Page

- Posts are being pulled from the database dynamically.
- Information about the post is displayed: thread, user, title, content.
- Posts can be interacted with by liking it (**only** for logged in users).
- Users can visit other pages from here: Login, Register and Profile pages.

Register Page

- New user can register to ThreadUD using their:
  Username, Email, Password, Year of Study and Name of the Course studying.
- Once pressed the Register button user is redirected to the home page and can now interact with the posts.
- Updates the database with new user and their information is stored safely.

Login Page

- Existing User can Log into their account using: Email and Password.
- Once Login button is pressed it searches database for a matching user.
- Once logged in the user is redirected to the home page and can now interact with the posts.

Profile Page

- View account information like: Name, Email, Year of Study and Course.
- Hides password for security purposes.
- Log out button that redirects you back to the Home page as a guest User once pressed.

Yet to implement (2nd semester)

Home Page

- Make a bottom nav-bar with icons for the other pages.
- Add comments icon to each post (displays number of comments) once pressed redirects the user to the Post page to make a comment.
- Add share icon to each post and implement a share to external feature.
- Add a join button where a user can join that thread from the Home page.
- Add a timestamp to each post that displays how long ago it was posted.
- Create an algorithm that makes it so that users only get posts most relevant to them based on their Year of study and the Course they are studying.

Thread Page

- Which will display only posts from that specific thread sorted newest>oldest.
- At the top it will say the Name of the Thread and the number of members.
- There will also be a join button at the top of the page.
- The posts will be pulled from the database and displayed to the screen in the same manner as on the Home page with the: timestamp, like, comment, share, Name of user that posted, content of post.
- You will also have a share icon at the top to share the Thread page.

Post Page

- Individual post page where that post will be displayed in a similar manner to Home & Thread pages but also having comments under it.
- User has the ability to make a comment or reply to a comment on that post.
- Able to also like comments or replies to comments by pressing on the icon.

Profile Page

- Can edit profile information for example changing password.
- View all the posts, comments or replies you've ever made on ThreadUD.

- Can edit Notifications preferences.

Search Page

- Users will be able to search for a specific Thread or topic.
- It will use key words to find what you are looking for in the database.

Make A Thread Page

- User can make a Thread with details such as: Name of Thread, Year of study targeted, Course(s) targeted, choose a banner and a bio for the Thread.
- This information will be saved in the database and users will now be able to find this Thread and make posts in it or read the posts in it.

Make A Post Page

- Here you will be able to make a post including: Title, optionally body text and images/videos and links to an external source.
- The User will also be able to select which Thread to post this in.

Notifications Page

- Implemented API that will sent notifications to the user's phone once user clicks on that notification they are redirected to the ThreadUD app on the Notifications page.
- Here they are able to view their notifications which could include: a post from a Thread they follow, someone liked their post, someone replied to their post, someone replied to a comment they made or someone liked a comment they made.
- Once you click on a notification it will redirect you to the page of that notification eg. Post page.

4. Detailed project plan for remaining work

Timeline for remaining work

Week	Tasks	Deadlines(end week or specified)
1 20th Jan	Post Thread/Home	
2 27th Jan	Post Thread/Home	
3 3rd Feb	Post Search Threads Make Post/Thread	
4 10th Feb	Search Threads Make Post/Thread	
5 17th Feb	Profile/Update Notifications	
6 24th Feb	Profile/Update Notifications	Pre API app done
7 3rd Mar	Moderator	
8 10th Mar	Moderator	
9 17th Mar	Notifications and verification	API app done
10 24th Mar	Cloud deployment Tests	
11 31st Mar	Tests	Open day 3rd
12 7th Apr	Correct failed tests	Testing done
13 14th Apr	Report	
14 21st Apr	Report	Report done
15 28th Apr	Presentation	END DATE 2nd

Task allocation breakdown

To ensure all parts of the project development are given sufficient time, the team will start working on the project the week before the start of semester 2 and work through the Easter break, giving the team 15 weeks to complete the full app.

During the development of the pre-api app, all front end development will be completed by Julia at the same time as each of the features back end development are being completed by either Nikita or David. Along side Julia, David will spend 2 and a half weeks developing the posts page back end, then 1 and a half weeks on the search threads page functionality. In the final two weeks of post api development, David will complete the profile and update profile page, while also finishing up any other parts of the pre-API app that are needed.

Meanwhile, Nikita will spend 2 weeks developing the thread and home page functionality, then 2 weeks on pages users will use to make posts and threads, spending the final 2 weeks developing the notifications page, although the development of this page could continue during the implementation of the notifications API

Next, the external APIs, including an ai moderator, notifications and verification for google log in will be added throughout the following 3 week period. This will include 2 weeks for the moderator API and 1 week for the remaining API. The implementation of these API will be done by all team members, as more research must also be done regarding their implementation.

Once the full app has been developed, it will be deployed to the cloud, including adding it to the google play store, deploying the backend database API to a hosting server and transferring the database to a larger database, with one team member completing each of these steps. Once this has been done, the team will begin testing the app using several automated and manual, black box and white box tests for the following 2 weeks. In the middle of the second week of testing, the team will present the pre testing pilot app at the TUD open day to get user feedback on the apps performance and function, as a final method of testing. Once all tests are complete, 1 week will be dedicated to fixing any issue arising from these tests. All work involved in testing and fixing issues will be split evenly between the 3 team members.

The final 3 weeks of the project have been dedicated to completing the project thesis repost and presentation, including the slides themselves and practicing presentations. However, if any steps in the apps development need more time for completion then initially planned for, additional time will be taken from this section of

the timeline to compensate. Provided the full 3 weeks are given for this section, the first 2 weeks will be spent writing the project thesis document, with the final week being dedicated to the creation of and practice of the project presentation.

5. Contributions

Project Overview – Julia

Use-Cases – all together

1. User Registration (Nikita)
2. User Login (Nikita)
3. View Home Page (Nikita)
4. Join a Thread (Nikita)
5. Create a New Thread (Nikita)
6. Make a Post (Nikita)
7. Like a comment (Nikita)
8. Comment on a Post (Julia)
9. Search for Threads (Julia)
10. Update Profile (Julia)
11. Receive Push Notifications (Julia)
12. Log Out (Julia)
13. Moderator Manages Posts (Julia)
14. View my posts (David)
15. View my comments (David)
16. Reply to a comment (David)
17. Like a post (David)
18. Share a post (David)
19. Forgot password (David)

Use-Case Diagrams – Nikita & Julia

- Figures 1, 2 and 3 (Julia)
- Figures 4, 5 and 6 (Nikita)

System Design – all

- The 4 High-end detailed Wireframes (Julia)
- System Diagrams (David)
- Detailed System Design implementation (Nikita)

Detailed project plan for remaining work – David

Document Layout & Contributions section – Nikita